

VPN Log Investigation in Splunk

Hunting for suspicious logins in VPN data

Overview

In this project I loaded VPN logs into Splunk and searched them for signs of account misuse. I built an index, checked the data, then looked for two common attack patterns: one account logging in from two countries too close together, and one IP address trying lots of different usernames. I finished by designing an alert for the spraying pattern.

Scenario

Northvale Logistics is a fictional UK company. Staff connect to the network through a VPN, and the VPN gateway writes a log entry every time a session starts or ends. The security team asked me to review a week of VPN logs and report anything unusual.

The company and its logs are made up. I used AI to help write a Python script that generates the week of logs, with normal staff activity and a hidden attack mixed in. This gave me realistic data to practise on without using any real company's logs.

Each log entry is in JSON and includes the username, source IP, source country, time and action. The action is login, login_failed or teardown, which is when a session ends.

```
{ "EventTime": "2026-09-14T08:02:11", "UserName": "j.harper",  
  "Source_ip": "198.51.100.23", "Source_Country": "United Kingdom",  
  "action": "login", "port": 443, "protocol": "tcp" }
```

Lab Setup

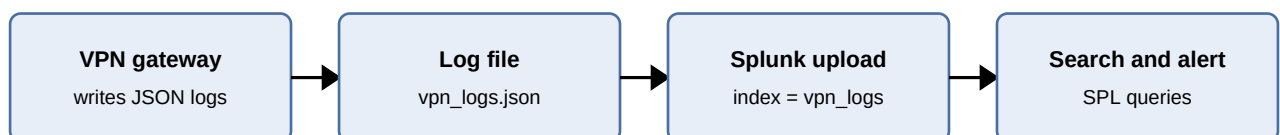


Figure 1. How the VPN data gets into Splunk

- Splunk Free running on my own machine.
- A new index called vpn_logs created under Settings, Indexes.
- The log file uploaded through Add Data, Upload, with the JSON source type.

The screenshot shows the 'New Index' configuration window in Splunk. Under 'General Settings', the 'Index Name' field is set to 'vpn_logs'. Below this field is a small lock icon and a note: 'Set index name (e.g., INDEX_NAME). Search using index=INDEX_NAME.' The 'Index Data Type' is set to 'Events' (indicated by a document icon), with 'Metrics' (indicated by a bar chart icon) as an alternative option. At the bottom, there is a small note: 'The type of data to store (event-based or metric)'.

Figure 2. Creating the vpn_logs index

Step 1: Check the Data

Before hunting, I checked the upload worked and the fields were parsed. Because I uploaded the file with the `_json` source type, Splunk pulls out the JSON fields like `UserName` and `Source_ip` on its own.

```
index=vpn_logs | stats count
```

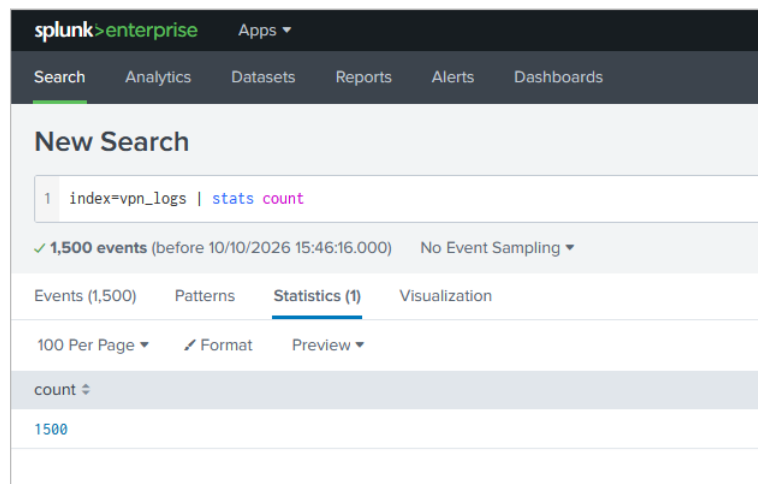


Figure 3. All 1,500 events loaded into the index

Next I broke the events down by country.

```
index=vpn_logs  
| stats count by Source_Country  
| sort - count
```

Source_Country	count
United Kingdom	1,412
Ireland	52
Brazil	28
Netherlands	8

Almost all logins come from the UK, which fits a UK company. Brazil stands out, since Northvale has no staff there.

Step 2: Impossible Travel

Impossible travel is when one account logs in from two places too far apart for the person to have moved between them. It often means someone else has the password. I followed up the Brazil logins by looking at which users they belonged to.

```
index=vpn_logs  
| search Source_Country="Brazil" action=login  
| stats count values(Source_ip) as ips by UserName
```

UserName	ips	count
----------	-----	-------

j.harper	203.0.113.45	1
----------	--------------	---

Only one login from Brazil succeeded, and it was j.harper. Then I put all of j.harper's logins that day in time order.

```
index=vpn_logs
| search Username="j.harper" action=login EventTime="2026-09-14*"
| table EventTime Source_ip Source_Country
| sort EventTime
```

The screenshot shows the Splunk Enterprise interface with a search query: `index=vpn_logs | search Username="j.harper" action=login EventTime="2026-09-14*" | table EventTime Source_ip Source_Country | sort EventTime`. The results are displayed in a table with 3 events:

EventTime	Source_ip	Source_Country
2026-09-14T08:02:11	198.51.100.23	United Kingdom
2026-09-14T08:14:37	203.0.113.45	Brazil
2026-09-14T13:30:05	198.51.100.23	United Kingdom

Figure 4. j.harper's logins on 14 September in time order

j.harper logged in from the UK at 08:02, then from Brazil at 08:14. Nobody can travel that far in 12 minutes, so the Brazil login is very likely not the real user. The next login at 13:30 is back in the UK, from their usual IP.

Step 3: Password Spraying

Password spraying is when an attacker tries one common password against lots of accounts, instead of lots of passwords against one account. A sign of it is one IP address trying many different usernames. I counted how many users each IP tried.

```
index=vpn_logs
| stats dc(Username) as users count by Source_ip
| where users > 3
| sort - users
```

Source_ip	users	count
203.0.113.45	9	28

To see what that IP actually did, I broke its events down by user and action.

```
index=vpn_logs
| search Source_ip="203.0.113.45"
| stats count by Username action
```

The screenshot shows a Splunk search interface with the following search query:

```
1 index=vpn_logs
2 | search Source_ip="203.0.113.45"
3 | stats count by Username action
```

The results table displays 28 events, showing the following data:

Username	action	count
a.patel	login_failed	3
d.walsh	login_failed	3
j.harper	login	1
j.harper	login_failed	2
j.harper	teardown	1
k.morgan	login_failed	3
l.bennett	login_failed	3
m.clarke	login_failed	3
r.hughes	login_failed	3
s.okafor	login_failed	3
t.ahmed	login_failed	3

Figure 5. Every event from 203.0.113.45, by user and action

Eight accounts had 3 failed logins each. j.harper had 2 failed logins, then a successful login and a session teardown. This is the same IP from the Brazil login, which links the two findings. It looks like an attacker sprayed passwords across many accounts, got into j.harper, then used the account.

Step 4: Planning an Alert

Finding this once is useful, but a SOC needs to catch it next time without someone searching by hand. The spraying search is a good fit for an alert. Splunk Free does not include alerting, so I could not build it in my lab. This is how I would set it up on a licensed Splunk instance.

```
index=vpn_logs action=login_failed
| stats dc(Username) as users count by Source_ip
| where users > 3
```

Setting	Value
Alert name	VPN Password Spraying
Schedule	Every hour, over the last hour
Trigger	Number of results is greater than 0
Action	Add to Triggered Alerts, severity High
Throttle	Once per IP every 24 hours, to avoid repeat alerts

I added `action=login_failed` to the alert search, so it only counts failed attempts. That cuts down false alarms from shared IPs, like an office network where many staff log in successfully.

Findings

Finding	Evidence	MITRE ATT&CK
Password spraying	203.0.113.45 tried 9 usernames	T1110.003 Password Spraying
Account compromise	j.harper logged in from UK then Brazil 12 minutes apart	T1078 Valid Accounts

Recommended Actions

- Reset j.harper's password and end any active VPN sessions.
- Block 203.0.113.45 at the VPN gateway.
- Check the other 8 accounts the IP tried, in case any were also accessed.
- Turn on multi-factor authentication for VPN logins.

What I Learned

- With the `_json` source type, adding `spath` extracts the fields a second time. That made every value show twice and inflated my counts, so I removed it.
- `stats dc( )` counts unique values, which is perfect for spotting one IP trying many accounts.
- Two small findings can link up into one bigger story, so it is worth following each lead.
- A good search should become an alert, so the same attack is caught automatically next time.

Skills Demonstrated

- Splunk index setup and data upload
- SPL searching and statistics
- Threat hunting in VPN logs
- Detecting impossible travel and password spraying
- Designing detection alerts
- MITRE ATT&CK mapping